# Session 5

# Table of contents


01 Recap & Building server with Express

02 How Client Sends Data

03 CRUD Operations

04 Project Structure

# 01

# Recap

# Recap

- What is the Backend
- What is Nodejs, Express, JS, TS
- what is a server-client connection
- what is HTTP & HTTP Methods
- What is an API

# What is Backend

- ★ The brain of the application
- ★ Handles data, logic, security
- ★ Talks to databases

# Backend Responsibilities

- ★ Authentication (login/signup)
- ★ Store & retrieve data
- ★ Business logic
- ★ Security & validation

# What is ..... ?!

## Nodejs

- It's a Runtime Environment
- Runs JavaScript on the server
- Fast & event-driven
- Used for backend

## Express

- It's a framework
- Simplifies server creation
- Handles routes & requests

## JS & TS

- They are just languages
- Ts may be considered as a layer above js that adds types

# Client – Server Connection

- Client → Request → Server
- Server → Response → Client

# Code with Me!

# What is HTTP?

- Communication protocol
- Request / Response
- Stateless

# HTTP Methods

- GET → Read
- POST → Create
- PUT → Update
- DELETE → Remove

# What Is an API?

- Application Programming Interface
- Contract between client & server
- Defines endpoints
- Defines inputs & outputs

# Types of APIs

- REST (most common)
- GraphQL
- SOAP
- gRPC.

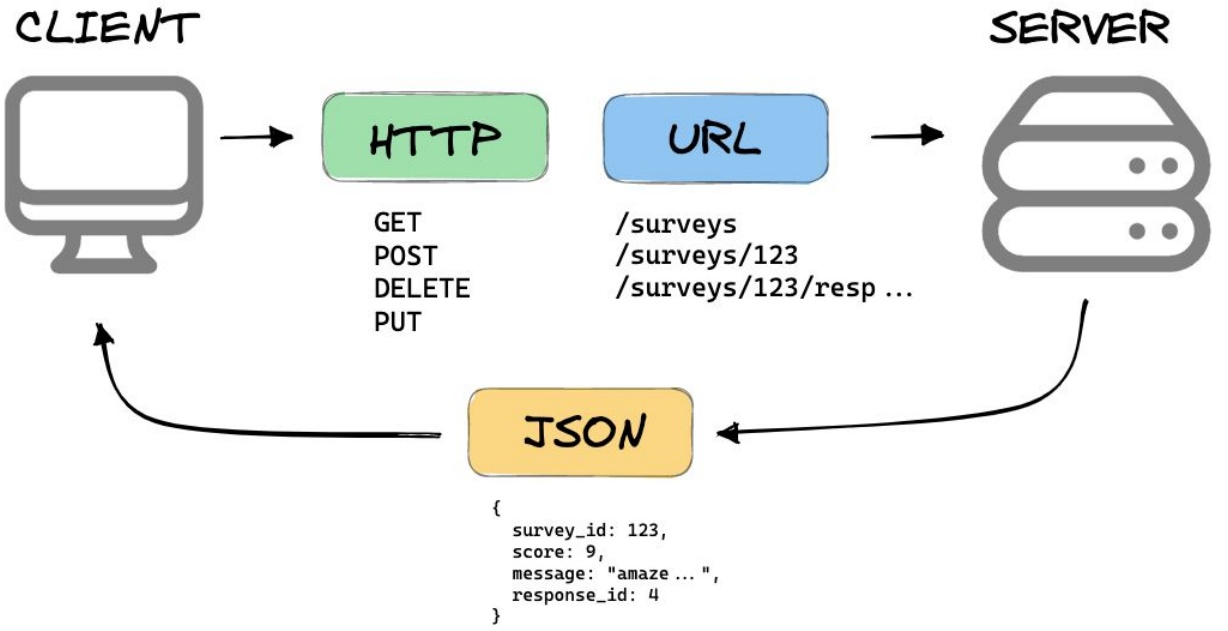
# Restful API

- REST is a design style for building APIs
- It defines how URLs, methods, and responses should be designed

# REST Core Ideas

- Resources, not actions
  - `/users` not `/getUsers`
- Use HTTP methods
  - `GET` → Read
  - `POST` → Create
  - `PUT` / `PATCH` → Update
  - `DELETE` → Remove
- Clear URLs
  - `/users`
  - `/users/5`
- Stateless
  - Each request is independent
- Standard responses
  - `200`, `201`, `404`, `500`

# WHAT IS A REST API?




mannhowie.com

# Code!

# Table of contents


~~01 Recap & Building server with Express~~

02 How Client Sends Data

03 CRUD Operations

04 Project Structure

# 02

# How Client Sends Data

# How Client Sends Data

Ways Client Sends Data:

- URL Params
- Query Params
- Request Body

# Code!

# Hands On

# Hands On

- Create a new GET endpoint
- This endpoint should accept a name sent from the client
- The server should respond with a welcome message must include the name sent by the client
- Test Your Endpoint ,Use Postman or your browser
- Try different names to make sure it works correctly

# Table of contents


~~01 Recap & Building server with Express~~

~~02 How Client Sends Data~~

03 CRUD Operations

04 Project Structure

03

# CRUD Operations

# CRUD Operations

| | | |
|---|---|---|
| ★ Create | | ★ POST |
| ★ Read | *Mapped to:* | ★ GET |
| ★ Update | | ★ PUT |
| ★ Delete | | ★ DELETE |

# Table of contents


~~01 Recap & Building server with Express~~

~~02 How Client Sends Data~~

~~03 CRUD Operations~~

04 Project Structure

# 04

# Project Structure

# Project Structure

## Why Structure Matters

- Maintainability
- Scalability
- Collaboration
- Debugging & Testing

## Recommended Project Structure

Below is the recommended structure for organizing your backend files:

```
📦 project-root
├── 📁 controllers  # Handles business logic
│   ├── fileController.js
│   ├── userController.js
│
├── 📁 models       # Data models (if applicable)
│   ├── fileModel.js
│   ├── userModel.js
│
├── 📁 routes       # API routes
│   ├── fileRoutes.js
│   ├── userRoutes.js
│
├── 📁 middlewares  # Middleware functions (e.g., validation, authentication)
│   ├── authMiddleware.js
│   ├── errorHandler.js
│
├── 📁 utils        # Utility functions (helpers)
│   ├── fileHelper.js
│   ├── responseHelper.js
│
├── 📁 config       # Configuration files (e.g., environment variables)
│   ├── config.js
│
├── server.js       # Entry point of the application
├── package.json    # Project metadata and dependencies
└── README.md       # Documentation
```

# Code with Me!

# Hands On

# Thanks!